1. AUREXION — TEST PLAN

1.1 Objective

The objective of testing is to verify that the Aurexion application works according to the defined requirements and that all major business workflows are functional, secure, and reliable.

The main areas include authentication/RBAC, CMS, CRM, RFP, Client Portal, Recruitment/ATS, Admin, API, security, responsive UI, and performance.

1.2 Scope

In Scope

- Login, logout and authentication
- Role-based access control
- Services, Industries, Case Studies and Blog
- Estimator
- RFP
- Contact/Lead management
- CRM
- Client Portal
- Projects and Documents
- Support Tickets
- Recruitment and ATS
- Admin functions
- REST APIs
- Database validation
- Security testing
- Responsive testing
- Browser compatibility
- Performance testing
- Regression testing

1.3 Testing Types

We will perform:

- Functional Testing
- UI Testing
- Validation Testing
- Positive Testing
- Negative Testing
- API Testing
- Integration Testing
- Database Testing
- RBAC Testing
- Security Testing
- Regression Testing
- Responsive Testing
- Cross-Browser Testing
- Accessibility Testing
- Performance Testing
- End-to-End Testing

The project specifically calls for security, API, database, performance and responsive validation in addition to functional testing.

1.4 Test Environment

Testing will be performed in the following:

- Frontend/Staging environment
- Backend/API environment
- Test database
- Test user accounts
- Required test data
- Supported browsers

Required roles should include the roles defined for the application so that RBAC can be verified.

1.5 Test Approach

Testing will follow this order:

Requirement Analysis
↓

Test Planning
↓
Test Case Preparation
↓
Environment Setup
↓
Smoke Testing
↓
Functional Testing
↓
API Testing
↓
Integration Testing
↓
Security/RBAC
↓
Performance
↓
Responsive & Browser Testing
↓
Automation
↓
Regression Testing
↓
Final E2E Testing
↓
Test Closure

1.6 Defect Management

When a defect is found:

Identify
↓
Reproduce
↓
Document
↓
Assign Severity/Priority
↓
Developer Fix
↓
Retest

↓
Pass → Close
Fail → Reopen

Defects will be categorized as:

- Critical
- High
- Medium
- Low

1.7 Entry Criteria

Testing starts when:

- Build is available
- Requirements are available
- Test environment is ready
- APIs are available
- Test credentials are available
- Required test data is available
- Major deployment blockers are resolved

1.8 Exit Criteria

Testing will be completed when:

- Planned test cases are executed
- Critical workflows pass
- API testing is completed
- RBAC/security testing is completed
- Regression testing is completed
- Critical and High defects are resolved
- Major E2E workflows pass
- Test report is prepared

The project acceptance criteria specify no Critical/High defects before approval.

Test Plan — Conclusion

The testing process will ensure that the Aurexion application meets its functional, security, usability, performance, and integration requirements. Testing will cover all major application modules and critical business workflows, including authentication and RBAC, CMS, CRM, RFP, Client Portal, Recruitment/ATS, Support, and Administration.

Testing will be performed progressively from **smoke testing → functional testing → API/integration testing → security and RBAC → performance → compatibility/accessibility → automation → regression → final end-to-end validation**.

All identified defects will be documented, tracked, retested, and verified before release. The application will be recommended for release only after critical business workflows have been successfully validated and **no Critical or High-severity defects remain open**, in accordance with the project's acceptance criteria.

2. AUREXION — TEST STRATEGY

2.1 Objective

The test strategy defines **how the Aurexion application will be tested** to ensure quality, functionality, security and reliability.

2.2 Testing Approach

We will use a **risk-based testing approach**.

High-risk modules will be tested first.

High-risk areas

1. Authentication
2. RBAC
3. Client data access
4. RFP
5. CRM
6. Client Portal
7. Documents

8. Recruitment/ATS
9. Support Tickets
10. APIs

The Client Portal requires object-level access restrictions so that one client cannot access another client's records, making authorization testing particularly important.

2.3 Manual Testing Strategy

Manual testing will cover:

- Functional scenarios
- Positive scenarios
- Negative scenarios
- Boundary conditions
- Validation
- UI behavior
- Error handling
- RBAC
- Business workflows
- Exploratory testing

2.4 API Testing Strategy

API testing will verify:

- HTTP methods
- Status codes
- Request parameters
- Request body
- Response body
- Authentication
- Authorization
- Validation
- Error handling
- CRUD operations
- Database persistence

Tools:

Postman / Swagger

2.5 Automation Strategy

Automation will be used for stable and repetitive scenarios.

API

Pytest

Automate:

- Authentication
- API smoke tests
- CRUD
- RBAC
- Critical business APIs
- Regression APIs

Frontend

Playwright

Automate:

- Login
- Critical UI workflows
- RFP
- CRM
- Client Portal
- Recruitment
- Support
- Regression
- E2E workflows

2.6 Security Strategy

Security testing will focus on:

- Authentication

- RBAC
- Unauthorized API access
- Object-level authorization
- SQL injection
- XSS
- CSRF/CORS
- Rate limiting
- File upload security
- Sensitive information exposure

These areas are explicitly identified in the project's security QA workflow.

2.7 Performance Strategy

Performance testing will focus on critical APIs and workflows such as:

- Login
- RFP
- CRM
- Client Dashboard
- Job Search
- Support Tickets

We will check:

- Response time
- Throughput
- Error rate
- Database/query performance
- Behavior under increased load

The project specifically calls for performance analysis including query counts, N+1 queries, indexes, pagination, caching and response-time measurement.

2.8 Compatibility Strategy

Test the application on:

Browsers

- Chrome
- Edge
- Firefox
- Safari/WebKit where available

Devices/Viewports

- Mobile
- Tablet
- Laptop
- Desktop

Check:

- Layout
- Navigation
- Forms
- Buttons
- Tables
- Modals
- Scrolling
- Touch interaction

2.9 Regression Strategy

Regression testing will be performed after:

- New features
- Bug fixes
- API changes
- Database changes
- Authentication changes
- Deployment

Priority:

Smoke
↓
Critical workflows
↓
Changed modules
↓
Integration

↓
Full regression

Test Strategy — Conclusion

The Aurexion testing strategy will follow a **risk-based and layered testing approach**, giving higher priority to authentication, authorization, client data protection, CRM, RFP, Client Portal, Recruitment/ATS, APIs, and other critical business workflows.

Manual testing will be used for functional, validation, exploratory, usability, and newly developed features. Automation will be used for stable and repetitive regression and end-to-end scenarios. API, database, security, performance, responsive, accessibility, and cross-browser testing will be performed as part of the overall quality process.

The final objective is to ensure that the application is **functionally correct, secure, reliable, responsive, maintainable, and ready for production release**.